CMU 15-849 分布式训练论文复现：集合通信、张量并行、ZeRO、流水线与重计算的模型与 RTX 5090 实测

ShaneLiu04*

2026 年 10 月

摘要

CMU 15-849（Machine Learning Systems，Zhihao Jia）是论文研讨课，没有官方编程作业。本报告对其分布式训练部分的必读论文（Ring/Rabenseifner all-reduce、Megatron-LM、ZeRO、GPipe/1F1B、Chen 2016/Checkmate 重计算）做最小复现：CPU 上用 numpy/torch 精确实现算法并用 α-β 与离散事件模型给出集群时间（71 个测试），再在一张 RTX 5090（32 GB，sm_120）上用真实 kernel 验证这些模型。主要结论：（1）GPT-2 124M/355M/774M 的静态训练显存实测为 16.00–16.32 B/参数（fp32、AMP、bf16+fp32 master 三种方案），与 `mlsys_calc.training_memory_bytes` 的 16N 一致，纯 bf16 为 8.01–8.31 B/参数；逐张量推导的激活模型（每层每 $s \cdot b \cdot h$ 保存 64/36/32 字节）与实测相差 0.4%–1.3%；（2）774M 上 bf16 训练步达到 70.2% MFU（相对 209.5 TFLOP/s 名义峰值；实测最佳 bf16 GEMM 为 234.7 TFLOP/s）；（3）每块全量重计算把激活压到 19.8%、最大 batch 从 8 提到 24，步时间只多 18.9%（理论 33%），只重算 MLP 用 9.3% 时间省 46% 激活（第 4.4 节）；ZeRO 分片与 4 级流水线的实测见第 4.5 节 and 4.6 节；（4）单卡上无法测 NVLink，本文实测 HBM 拷贝 1518 GB/s、PCIe 5.0 H2D 56.5 GB/s、kernel 启动约 10 µs，并用它们给 α-β 模型定参：在无 NVLink 的 8×5090 PCIe 机器上，774M 的 DP 梯度 all-reduce 约需 48 ms/步，是 H100 NVLink 的 8 倍。

目录

1 背景与作业要求 **3**

2 实验环境 **3**

3 方法与实现 **3**

3.1 CPU 部分（已有实验） 3

3.2 GPU 部分：模型与测量方法 4

3.3 测试 5

4 实验结果 **6**

4.1 CPU：all-reduce 算法与 gloo 实测 6

4.2 CPU：ZeRO、流水线与重计算模型 7

4.3 GPU A1：训练显存逐组件核算 9

4.4 GPU A2：激活重计算 11

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

4.5 GPU A3：ZeRO 分片的每 rank 显存与通信参数 13
4.6 GPU A4：真实 kernel 的 GPipe 与 1F1B 15

5 与理论模型的对照 **17**
5.1 静态显存：16N 的三种拆法 17
5.2 激活：逐张量计数 17
5.3 训练步时间与 MFU 18
5.4 α-β 模型的单卡定参 18
5.5 流水线气泡：均衡与不均衡 18
5.6 重计算的时间与显存 18
5.7 $\sqrt{n}$ 与 Checkmate 19

6 面试专题 **19**

7 局限 **21**

8 结论 **21**

1 背景与作业要求

15-849 的分布式训练专题围绕四类并行与一类显存优化展开。由于没有官方作业，我们把每篇论文的“核心可检验命题”映射成实验题：

表 1: 论文、命题与本实验的对应关系

论文	可检验命题	本实验
Ring all-reduce / Rabenseifner [1, 2]	每 rank 发送 $2(P-1)/P \cdot S$；步数 $2(P-1)$ vs $2\log_2 P$	CPU 模拟 + gloo 实测；单卡 ring 仿真定参 α, β
Megatron-LM [3]	列切 → 行切 MLP 每块前/反向各 1 次 all-reduce	CPU 精确等价（fp64 误差 $\leq 8.9 \times 10^{-16}$）
ZeRO [4]	每卡状态 $16\Psi \to (4+12/N_d)\Psi \to \cdots \to 16\Psi/N_d$	CPU 4 rank 模拟；GPU 上真实分片张量测显存、分片 AdamW 等价
GPipe [5] / PipeDream-1F1B [6, 7]	气泡 $(p-1)/(m+p-1)$；1F1B 在途激活 $\leq p$	离散事件模拟；GPU 上 4 级 GPT 真 kernel 执行 + CUDA event 时间线
Chen 2016 [8]/Checkmate [9]	$\sqrt{n}$ 分段；全重计算时间 $\approx 4/3$	CPU DP 前沿；GPU 上 `torch.utils.checkpoint` 全量/选择性
混合精度 [11]	Adam 混合精度 16 B/参数（2+2+12）	GPU 上 fp32/AMP/bf16/bf16+master 逐组件测量

GPU 部分按任务说明分为四题：A1 训练显存核算（权重/梯度/优化器/激活，三种精度，16N 规则，激活随 $b \times s$）；A2 激活重计算（每块 checkpoint 与选择性 checkpoint，显存节省 vs 时间开销，最大可放 batch）；A3 单卡模拟 ZeRO 各 stage 的每 rank 显存，并测量可用于 all-reduce 代价模型的带宽参数；A4 4 级流水线 GPipe 与 1F1B 的真实执行时间线、气泡与峰值激活，以及训练步 MFU。

2 实验环境

表 2: 软硬件环境（GPU 信息取自 `hw.jsonl` 的 env 行）

本地	Windows Server，4 物理核；Python 3.12，torch CPU；CPU 部分全部在此运行
GPU 服务器	AutoDL，NVIDIA GeForce RTX 5090，32 GB GDDR7，170 SM，sm_120，96 MB L2
GPU 软件	PyTorch 2.8.0+cu128，CUDA 12.8，Python 3.12.3
共享约束	7 个代理共用一卡：所有 GPU 命令在 `flock` 互斥锁下串行，CPU 限 2 核（`taskset -c 44,45`）
名义峰值	bf16 稠密（fp32 累加）209.5 TFLOP/s；GDDR7 1.792 TB/s；PCIe 5.0 x16

互斥锁保证计时时 GPU 上没有别的作业；但 CPU 只有 2 核，Python 端开销比独占机器略大，这对小 kernel、启动受限的测量（流水线小算子、单卡 ring）影响最大。

3 方法与实现

3.1 CPU 部分（已有实验）

`code/disttrain/` 下六个模块：`collectives.py`（naive、ring RS+AG、递归减半-倍增、二项树，真实搬 numpy 数组并逐 rank 计字节与步数，支持交换机与物理环两种拓扑）；

`tensor_parallel.py`（显式 f/g 算子的列/行并行 MLP 与按头切分 attention）；`zero.py`（stage 0–3、ring RS/AG、分片 Adam）；`pipeline.py`（固定每卡算子序的离散事件模拟，GPipe/1F1B/交错）；`checkpointing.py`（链上 no-ckpt/$\sqrt{n}$/预算约束 DP，并用 torch checkpoint 验证）；`gloo_bench.py`（2/4 进程 gloo all-reduce 实测）。

3.2 GPU 部分：模型与测量方法

模型。 `gpu/gpt.py` 是 nanoGPT 风格的 GPT-2（tied embedding，dropout=0，attention 用 `F.scaled_dot_product_attention`，CUDA 上走 flash/mem-efficient kernel，因此不保存 s^2 的注意力分数）。参数量公式 `GPTConfig.params()` 与 HF GPT-2 124M 的 124,439,808 一致（测试）。每个 `Block` 有 `ckpt` 属性：`none`/`full`（整块重算）/`mlp`（只重算 MLP 子层）/`attn`（只重算 attention 子层）；`Stage` 把连续若干块切成流水线的一级，首级含 embedding，末级含 LM head 与 loss。

Listing 1: 每块重计算模式（gpu/gpt.py）

```python
def forward(self, x):
   if self.ckpt == "full" and self.training:
      return checkpoint(self._body, x, use_reentrant=False)
   if self.ckpt == "attn" and self.training:
      x = x + checkpoint(self._attn, x, use_reentrant=False)
   else:
      x = x + self._attn(x)
   if self.ckpt == "mlp" and self.training:
      return x + checkpoint(self._mlp, x, use_reentrant=False)
   return x + self._mlp(x)
```

四种精度方案。 **fp32**：全 fp32 + fused AdamW；**AMP bf16**：fp32 权重/梯度/状态，前向在 `torch.autocast(bfloat16)` 下；**纯 bf16**：权重、梯度、Adam 状态全 bf16；**bf16+fp32 master**：bf16 模型 + fp32 主权重副本，AdamW 作用在主权重上，步后拷回——这就是 ZeRO 论文与 `mlsys_calc` 假设的“2+2+12”方案。

逐组件显存测量。 用 `torch.cuda.memory_allocated` 在训练步的边界取差（listing 2）：建模型后的增量 = 权重；前向后（loss 仍持有计算图）的增量 = 保存的激活；反向后激活已释放、梯度已分配，增量 = 梯度；第一次 `opt.step()` 惰性分配 Adam 状态，步后（梯度置 None）常驻量减权重 = 优化器状态（含 master）。峰值用 `max_memory_allocated`，在稳态步上取。

Listing 2: 训练步内的显存采样（gpu/train_bench.py，节选）

```python
m0 = alloc(); torch.cuda.reset_peak_memory_stats()
with torch.autocast("cuda", dtype=torch.bfloat16): # AMP 模式
   loss = model(x, y)
rec["act"] = alloc() - m0 # 保存的激活（含 logits 相关）
rec["peak_fwd"] = torch.cuda.max_memory_allocated() - m0
loss.backward()
rec["grads"] = alloc() - m0 # 激活已释放，梯度已分配
...
opt.step() # 第一次调用分配 exp_avg / exp_avg_sq
```

计时与 MFU。 训练步计时为 `synchronize` 包围的 wall time，取 2 次预热后 10 次中位数。每 token FLOPs 取 $6N + 12Lhs$（后者是 $QK^\top$ 与 PV 的前 + 反向，未按因果减半），MFU = tokens/s × FLOPs/token/209.5 TFLOP/s。

ZeRO 单卡模拟。 把 774M 模型的全部参数拼成一维 bf16 张量（$N = 774\,030\,080$），按 DeepSpeed 的做法补齐到 N_d 的倍数后切片。对每个 (N_d, stage) 真实分配 rank 0 在该 stage 下应持有的张量（bf16 权重 N 或 N/N_d，bf16 梯度 N 或 N/N_d，fp32 master/m/v 各 N 或 N/N_d），用 `memory_allocated` 计量。另在 $N_d \in \{1, 4, 16\}$ 下逐 rank 执行分片 fused AdamW 两步，拼回后与整张量 AdamW 比较（检验 ZeRO-1 的数学等价）并计时。

带宽与 α-β 定参。 单卡没有 NVLink，也无法做真实跨卡 all-reduce。我们测四类“链路”：D2D 拷贝（HBM 读 + 写）、`y.add_(x)` 归约（2 读 1 写）、pinned H2D/D2H（PCIe 5.0，多卡 5090 机器没有 NVLink 时 all-reduce 的真实瓶颈），以及“单卡 ring all-reduce”：P 个缓冲区在同一卡上按 ring 的 RS+AG 顺序做 $2(P-1)\cdot P$ 次块拷贝/加法。后者的小消息时间测得的是*步数* $\times$*kernel 启动*，即 α 的单卡下界；大消息测得的是 HBM 侧的 β。

流水线实测。 4 级 GPT-2 355M（每级 6 块，micro-batch $b = 4$, $s = 512$）。单卡上 4 个 stage 不能真正并行，所以分两步：（1）用 lab 的 `schedule_orders` 生成每级的 F/B 顺序，以单位代价重放得到一个与并行调度一致的串行化顺序，在 GPU 上按此顺序执行真实前/反向，每个算子前后打 CUDA event 得到实测时长，并在每个 F 后记录该级该 micro-batch 新增的激活字节；（2）用实测时长按同样的依赖规则（F 等上一级 F，B 等下一级 B）重放得到*实测时长驱动的* Gantt 图、makespan 与气泡。峰值显存分两种口径：单卡上 4 级合计的 `max_memory_allocated`（真实执行），以及每级在途 micro-batch 数 $\times$ 该级每 micro-batch 激活字节。

Listing 3: 流水线算子执行（gpu/train_bench.py，节选）

```
if k == "F":
    inp = xs[i] if d == 0 else acts[(d - 1, i)][1].detach().requires_grad_()
    with torch.autocast("cuda", dtype=torch.bfloat16):
        out = stages[d](inp, ys[i] if d == p - 1 else None)
    acts[(d, i)] = (inp, out)
else:
    inp, out = acts.pop((d, i))
    out.backward() if d == p - 1 else out.backward(grads.pop((d, i)))
    if d > 0:
        grads[(d - 1, i)] = inp.grad # "发送"给上一级的梯度
```

3.3 测试

CPU 部分 71 个用例保持不变。新增 `tests/test_gpu_training.py`：参数量公式、FLOPs/激活公式、三种 checkpoint 模式梯度与不 checkpoint *逐位相等*（fp64，CPU）、4 个 Stage 串联与完整模型逐位相等、实测时长重放在单位代价下与 lab 模拟器及 $(p-1)/(m+p-1)$ 一致；CUDA 上：四种精度训练步有限且组件为正、GPipe 与 1F1B 执行得到的梯度一致（atol $= 10^{-5}$）。无 CUDA 时 CUDA 用例 skip。

4 实验结果

4.1 CPU：all-reduce 算法与 gloo 实测

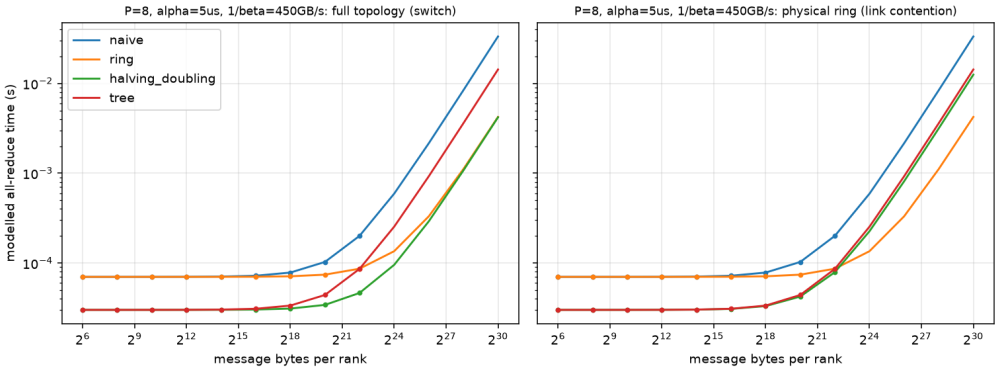


图 1: P=8 时四种 all-reduce 的模型时间（$\alpha = 5$ µs 假设，β 取 H100 NVLink 450 GB/s）。

图 1 是 `collectives.py` 的输出：模拟器逐 rank 搬运数组并计字节，时间与闭式公式一致到 10^{-12}。1 KB 时 ring 70.0 µs、halving-doubling 与二项树 30.0 µs——小消息由步数（$2(P-1) = 14$ vs $2\log_2 8 = 6$）决定；1 GB 时 ring 4.25 ms、halving-doubling 4.21 ms（交换机），但在物理环上 halving-doubling 因远距离配对的链路争用升到 12.56 ms。ring 超过树的交叉点约 4.27 MB。含义：NCCL 同时实现 ring 与 tree 并按消息大小选择，不是冗余。

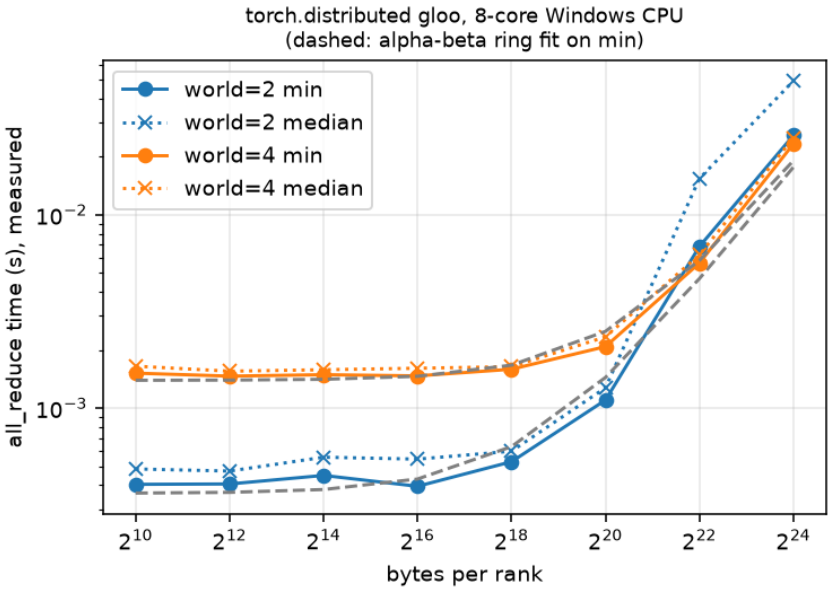


图 2: 本机 CPU gloo all-reduce 实测（20 次取最小）。

图 2：world=2 时 1 KB 0.40 ms、16 MB 25.8 ms；按 ring 公式拟合 $\alpha \approx 181$ µs、等效带宽 0.97 GB/s（world=4：232 µs、1.42 GB/s）。它只说明“同一个公式能拟合真实实现”，数值比 NVLink 慢两到三个数量级，不能外推。

4.2 CPU：ZeRO、流水线与重计算模型

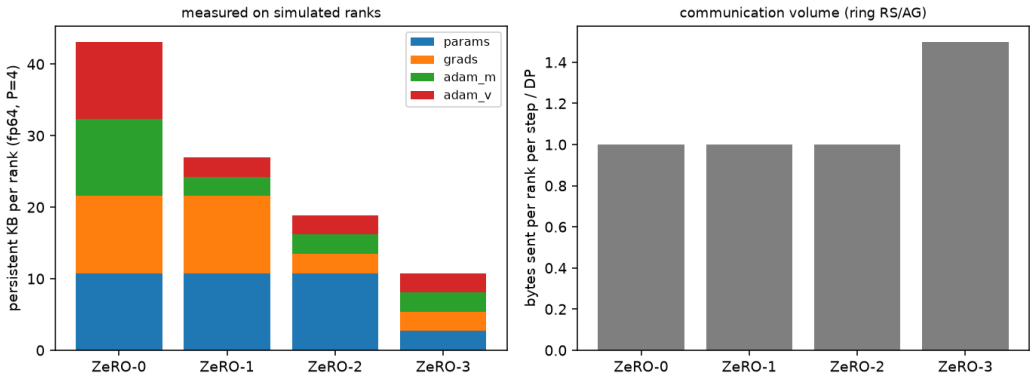


图 3: ZeRO stage 0–3：P=4 模拟的每 rank 常驻字节与通信量（左），论文规模 $\Psi = 7.5\text{B}$、$N_d = 64$ 的 `mlsys_calc` 值（右）。

图 3：fp64 模拟中四个 stage 与单进程训练最大差 2.9×10^{-16}，每 rank 常驻字节 43,136 / 26,960 / 18,872 / 10,784，即相对 DP 为 1 / 0.625 / 0.4375 / 0.25，与 (1)、$(2+2/P)/4$、$(1+3/P)/4$、$1/P$ 吻合；ZeRO-3 通信为 DP 的 1.5 倍。按论文混合精度、$\Psi = 7.5\text{B}$、$N_d = 64$ 得 120 / 31.4 / 16.6 / 1.88 GB，与 ZeRO 论文图 1 一致。

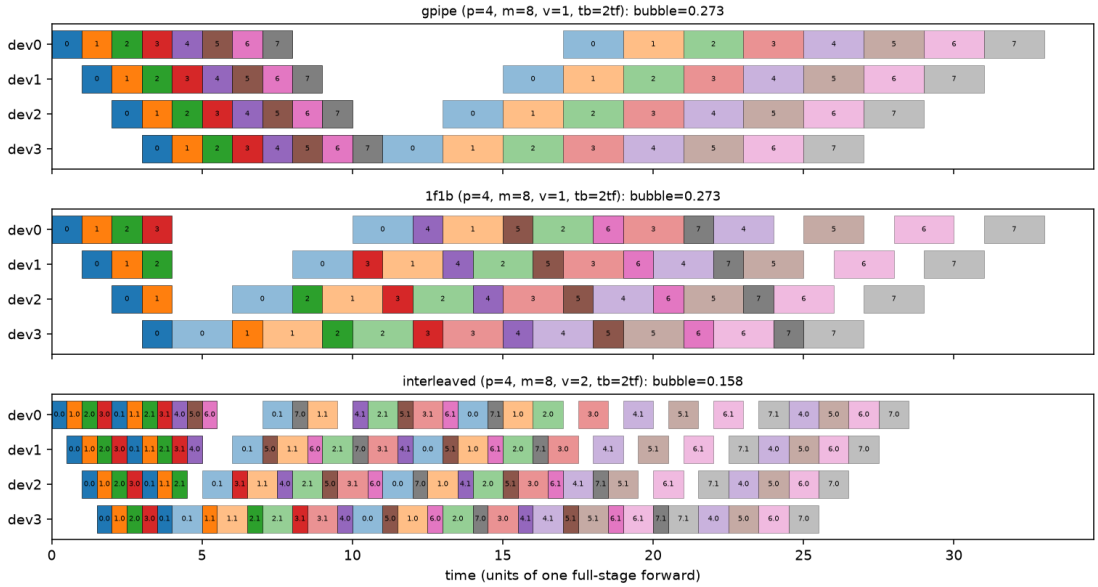


图 4: 离散事件模拟的 GPipe / 1F1B / 交错调度（p=4, m=8，$t_b = 2t_f$）。

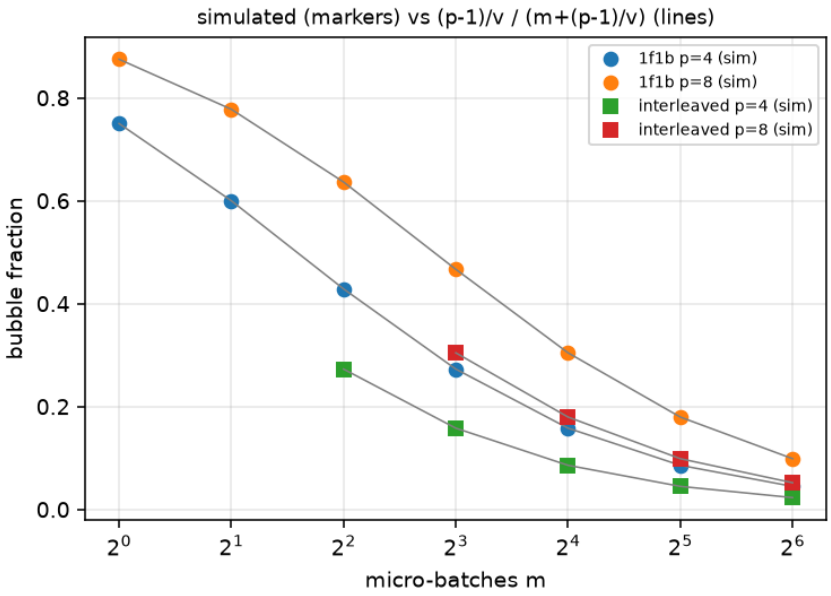


图 5: 模拟气泡随 m 的变化与 $(p-1)/(m+p-1)$。

图 4 and 5：均匀算子下 GPipe 与 1F1B 的气泡完全相同（p=4, m=8 都是 0.273），1F1B 只把设备 0 的在途激活从 $m=8$ 降到 $p=4$；交错 $v=2$ 把气泡降到 0.158，但设备 0 的在途激活变成 5.5（比 1F1B 多）。

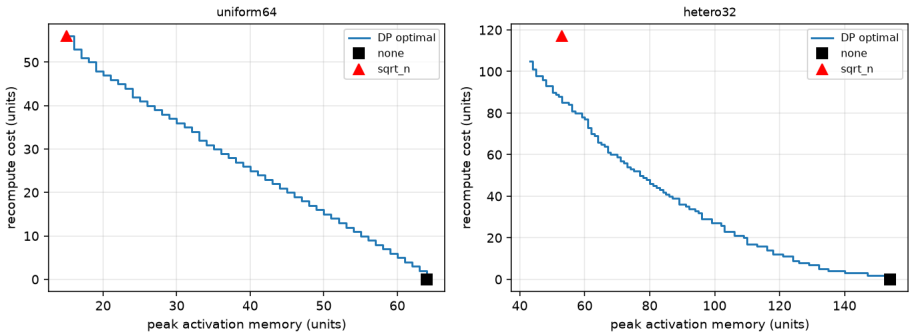


图 6: 重计算的显存-重算 Pareto 前沿：均匀 64 层与异构 32 层链。

图 6：均匀 64 层时 $\sqrt{n}$（每 8 层一段）峰值 15、重算 56，已在 DP 前沿上；异构链上同预算 53，$\sqrt{n}$ 重算 117，DP 重算 85（少 27%）。CPU 上 torch 16 层小 MLP 的时间比为 1.34–1.73，远高于理论 4/3，原因是 Python 钩子与小算子开销——这正是需要 GPU 实测的地方。

4.3 GPU A1：训练显存逐组件核算

表 3: 训练显存逐组件实测（`memory.jsonl`；GB=10^9 字节；B/p = (权重 + 梯度 + 优化器)/参数数；激活模型见第 5.2 节）

模型	方案	权重	梯度	优化器	B/p	激活实测	激活模型	峰值	tokens/s	MFU
gpt2-124m	fp32	0.50	0.52	1.01	16.30	6.54	6.48	11.34	54.5k	22.4%
gpt2-124m	AMP bf16	0.50	0.51	1.01	16.19	4.67	4.61	9.46	125.0k	51.3%
gpt2-124m	纯 bf16	0.26	0.26	0.52	8.31	4.09	4.06	8.17	131.1k	53.8%
gpt2-124m	bf16+master	0.26	0.26	1.51	16.32	4.09	4.06	9.16	127.9k	52.5%
gpt2-355m	fp32	1.42	1.42	2.84	16.00	14.61	14.53	22.17	20.5k	23.8%
gpt2-355m	AMP bf16	1.42	1.42	2.84	16.00	9.67	9.60	17.22	53.0k	61.6%
gpt2-355m	纯 bf16	0.71	0.71	1.42	8.01	8.14	8.09	13.56	56.6k	65.6%
gpt2-355m	bf16+master	0.71	0.71	4.26	16.01	8.14	8.09	16.40	55.2k	64.0%
gpt2-774m	fp32	3.17	3.10	6.19	16.10	12.96	12.90	23.97	9.5k	23.7%
gpt2-774m	AMP bf16	3.17	3.10	6.22	16.14	9.23	9.16	20.30	26.3k	65.3%
gpt2-774m	纯 bf16	1.58	1.59	3.16	8.17	6.90	6.86	13.28	28.2k	70.2%
gpt2-774m	bf16+master	1.58	1.57	9.47	16.31	6.90	6.86	19.60	26.8k	66.7%

124M/355M 为 $b = 8, s = 1024$，774M 为 $b = 4, s = 1024$。

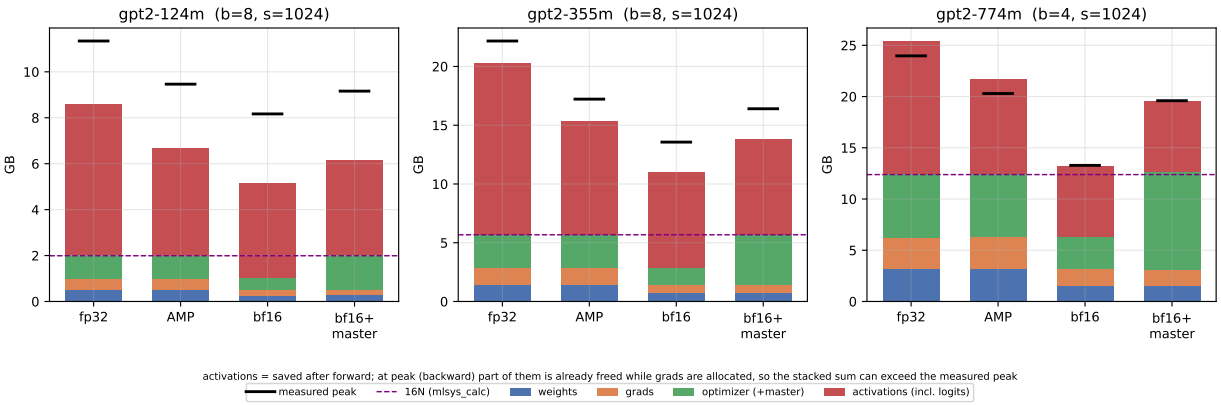


图 7: 逐组件显存堆叠图与稳态峰值（黑线），紫色虚线为 16N。

图 7 与表 3 说明三件事。第一，**“16N”不是混合精度独有**：fp32（4+4+8）、PyTorch AMP（fp32 权重 4 + fp32 梯度 4 + m/v 8）和 bf16+fp32 master（2+2+12）三种完全不同的布局都落在 16.00–16.32 B/参数；只是分配方式不同——AMP 的 16 字节里没有 bf16 权重，而 bf16+master 方案的优化器一项就是 12N（774M：9.47 GB）。超出 16 的 0.0–0.3 字节来自 CUDA 缓存分配器 512 B 粒度取整、LayerNorm/bias 这类小张量以及 fused AdamW 的 step 计数张量。第二，**纯 bf16** 只有 8.01–8.31 B/参数，但它把 Adam 的 m/v 放在 bf16 里，长时间训练会因 bf16 尾数只有 7 位而在小更新上“吞掉”增量（参见第 7 节），这就是工业界仍用 fp32 master 的原因。第三，**激活才是大头**：在这些 batch 下激活是静态状态的 1.5–5.7 倍；AMP 比 fp32 省激活（9.67 vs 14.61 GB，355M），却不省静态显存。

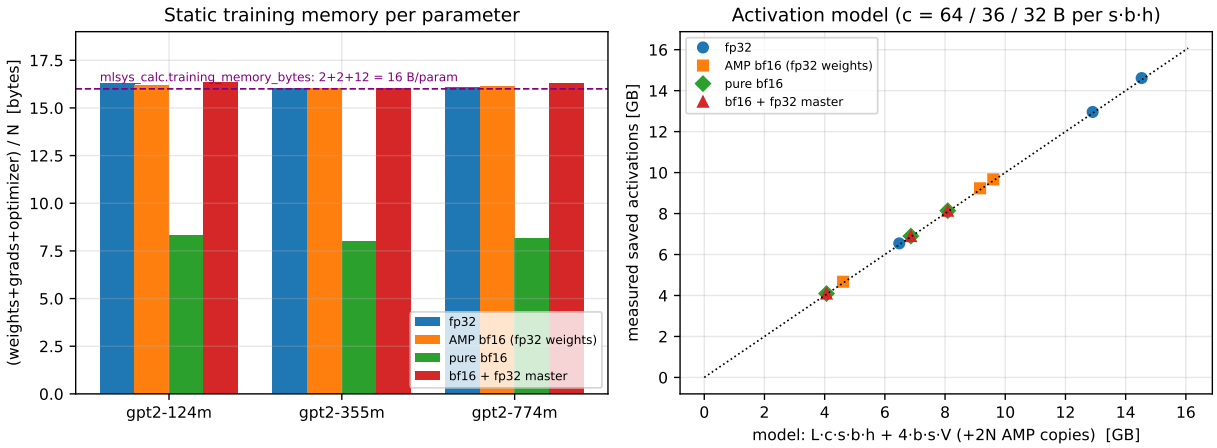


图 8: 左：静态显存/参数，与 `mlsys_calc` 的 16 B/参数对照；右：激活模型 vs 实测（12 个配置）。

图 8 右图把第 5.2 节 的逐张量模型（每层 $c \cdot sbh$，$c = 64/36/32$；加 cross-entropy 的 fp32 log-softmax 输出 $4bsV$；AMP 再加 autocast 为反向保存的 bf16 权重副本 $\approx 2N$）与实测对齐：12 个配置的实测/模型比为 1.004–1.013。残差约 1%，来自 embedding 输出、位置索引与 loss 标量等未计入的小张量。

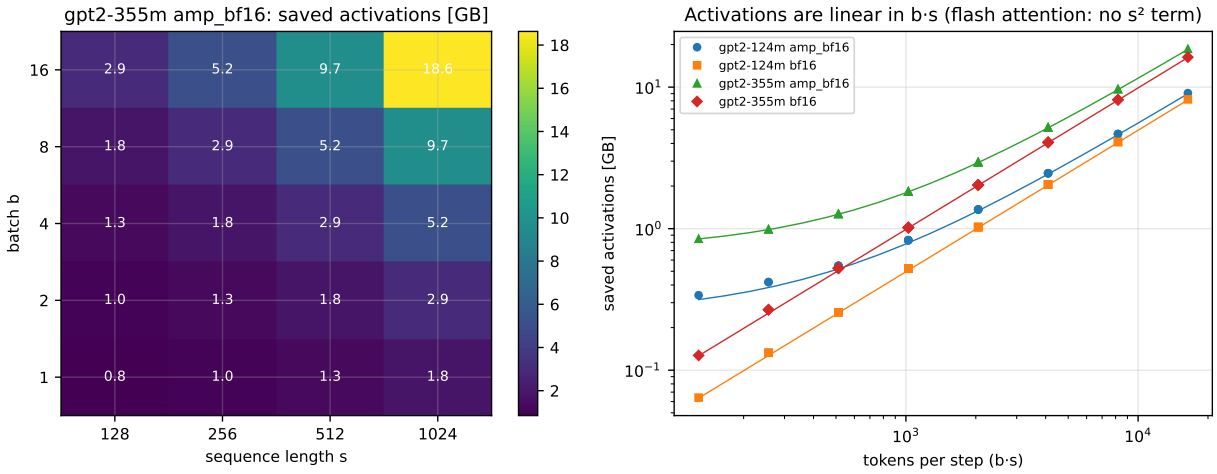


图 9: 左：355M AMP 的激活随 $b \times s$ 的热力图；右：对数坐标下激活与 $b \cdot s$ 成正比（线为模型）。

图 9：激活对 $b \cdot s$ 严格线性，没有 s^2 项（flash attention [12] 不保存注意力矩阵）。线性拟合斜率：124M AMP 536,843 B/token（模型 532,804），355M AMP 1,093,862 B/token（模型 1,085,764）；截距 272 MB / 709 MB 对应 AMP 的 bf16 权重副本（模型 247 MB / 707 MB）；纯 bf16 的截距仅 4–6 MB，正好印证“截距 = autocast 权重缓存”。值得注意的是每 token 的激活里 logits 项 $4V = 201{,}028$ 字节占了 124M 的 38%：词表大、模型小时，loss 那一层就是激活大头，这也是 Megatron/Liger 等实现“分块 cross-entropy”的动机。

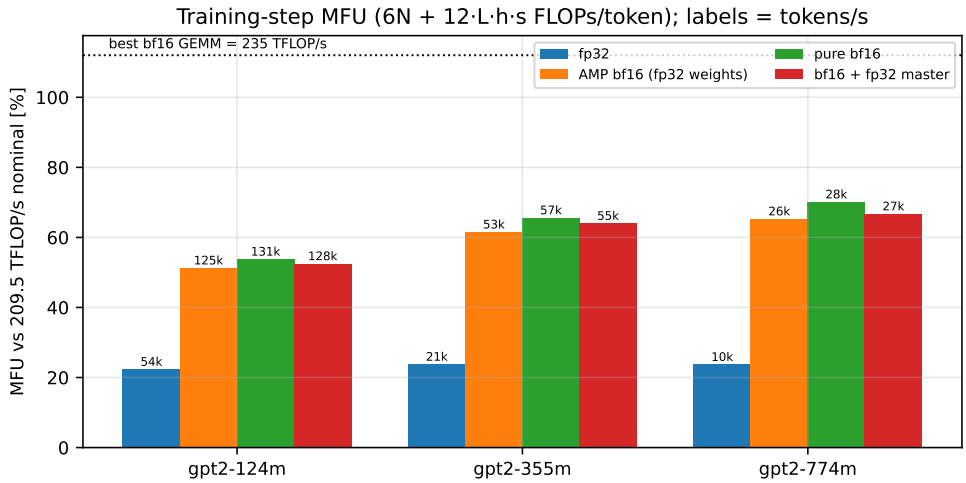


图 10: 训练步 MFU（相对 209.5 TFLOP/s），柱上数字为 tokens/s；虚线为实测最佳 bf16 GEMM。

图 10：bf16 类方案的 MFU 随模型变大而升高（124M 51–54%，355M 62–66%，774M 65–70%），因为 d 越大 GEMM 越接近峰值、LayerNorm/GELU 等访存型算子占比越小；fp32 只有 22–24%（不用 tensor core 的 bf16 路径，走 TF32 被关闭时的 fp32 FMA）。实测 8192^3 bf16 GEMM 为 234.7 TFLOP/s，高于名义 209.5——GeForce Blackwell 的 fp32 累加并未像上一代那样减半，因此若以实测 GEMM 为分母，774M 纯 bf16 的 MFU 为 $70.2\% \times 209.5/234.7 = 62.7\%$。bf16+master 比纯 bf16 慢约 5%，就是每步多出的 12N 字节主权重读写与 fp32 梯度转换。

结论：显存核算

16N 规则在三种布局下都成立（实测 16.00–16.32 B/参数），纯 bf16 为 8 B/参数；逐张量激活模型 $L \cdot c \cdot sbh + 4bsV$ $(+2N)$ 误差 $\leq 1.3\%$。在 GPT-2 规模上，决定能否放下的是激活，而不是参数。

4.4 GPU A2：激活重计算

774M、AMP、$b = 8$、$s = 1024$，对五种策略各测一次显存与步时间（`recompute.jsonl`；`expandable_segments` 分配器），再对四种策略在 $\{4, 8, 12, 16, 20, 24, 32, \ldots\}$ 的阶梯上找最大可放 batch（遇到第一次 OOM 即停）。

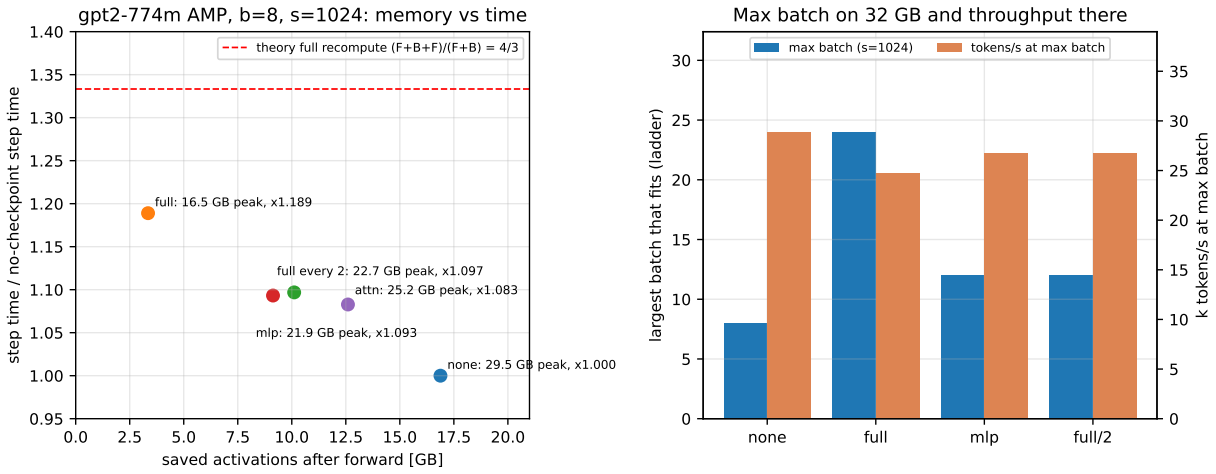


图 11: 左：前向后保存的激活 vs 步时间（相对不 checkpoint）；右：32 GB 上的最大 batch 与该 batch 下的吞吐。

表 4: 重计算的显存与时间（774M，AMP，$b = 8$，$s = 1024$，`recompute.jsonl`）

策略	保存激活 (GB)	相对	峰值总显存 (GB)	步时间 (ms)	时间比
不 checkpoint	16.880	1.000	29.47	282.6	1.000
每块全量	3.348	0.198	16.54	336.1	1.189
隔块全量（every 2）	10.110	0.599	22.71	310.0	1.097
只 MLP	9.131	0.541	21.85	309.0	1.093
只 attention	12.599	0.746	25.20	306.0	1.083

显存与逐张量模型逐项吻合。 用第 5.2 节 的计数（AMP，每层 $c = 36$ 字节/sbh，$L \cdot sbh = 36 \times 8 \times 1024 \times 1280 = 3.775 \times 10^8$）：

- 不 checkpoint：$36 \cdot L\, sbh + 4bsV + 2N = 13.59 + 1.647 + 1.548 = 16.785$ GB，实测 16.880 GB（比 1.006）。
- 只 MLP：省掉 MLP 内部 $2 + 8 + 8 = 18$ 字节/sbh（6.795 GB），再加 autocast 为 `fc`/`fc2` 保存的 bf16 权重副本 $8h^2 \cdot 2 \cdot L = 0.944$ GB，合计 7.739 GB；实测节省 $16.880 - 9.131 = 7.749$ GB。
- 只 attention：checkpoint 的输入（fp32 残差，4 字节）仍要保存，只省 LN 输出 2 + QKV 6 + attention 输出 2 = 10 字节（3.775 GB），加 `qkv`/`proj` 权重副本 0.472 GB，合计 4.247 GB；实测节省 4.281 GB。
- 每块全量：每层只剩块输入 $4sbh$（1.510 GB）加 $4bsV$（1.647 GB），共 3.157 GB；实测 3.348 GB，多出的 0.19 GB 恰好是最后一层 LN 的 fp32 输入（42 MB）、其 bf16 输出（21 MB）与 LM head 的 bf16 权重副本（$Vh \cdot 2 = 129$ MB），这些在 checkpoint 区域之外。

所以“只 MLP”是性价比最高的选择性方案：每层 36 字节中有一半在 MLP（GELU 的输入与输出都是 $4h$ 宽）。

时间远低于 4/3。 每块全量重计算只慢 18.9%，不是理论的 33.3%。4/3 的推导假设整步只有块的前向与反向且 $B = 2F$；实测时间比说明“块前向”只占整步的 18.9%。差额来自三处不被重算的工作：LM head 与 cross-entropy（$V = 50257$ 时 LM head 约占前向 FLOPs 的 8%）、fused AdamW（读写 fp32 参数、梯度与两个矩，约 21.7 GB 访存），以及反向比 $2F$ 更贵的部分（flash attention 反向内

部本身要重算注意力分数，AMP 下还有梯度的 dtype 转换）。选择性方案的时间比为 1.083–1.097，与它们重算的 FLOPs 比例同序：MLP 占块 FLOPs 约 2/3，却只多 9.3%，因为重算的是 GEMM（效率最高的部分）。

最大 batch 与吞吐。 32 GB 上不 checkpoint 最多 $b = 8$（28.9k tokens/s，MFU 71.8%）；每块全量重算可放 $b = 24$（3 倍），但吞吐降到 24.7k tokens/s（MFU 61.4%）；只 MLP 与隔块全量都是 $b = 12$，约 26.7–26.8k tokens/s。在 $b = 8$ 时 GPU 已经饱和（MFU 72%），更大的 batch 不再提高效率，所以重计算换来的显存没有被吞吐收益抵消——它的价值在于让放不下的配置（更长序列、更大模型、或需要更大全局 batch 而不想用梯度累积）变得可行。

结论：重计算

全量重计算把激活压到 19.8%、最大 batch 提高 3 倍，时间只多 18.9%（理论 33%，因为 LM head、优化器与更贵的反向不被重算）；只重算 MLP 用 9.3% 的时间省 46% 的激活，节省量与逐张量模型（含 autocast 权重副本）相差 0.1%。GPU 已饱和时，重计算不会因为 batch 变大而“回本”。

4.5 GPU A3：ZeRO 分片的每 rank 显存与通信参数

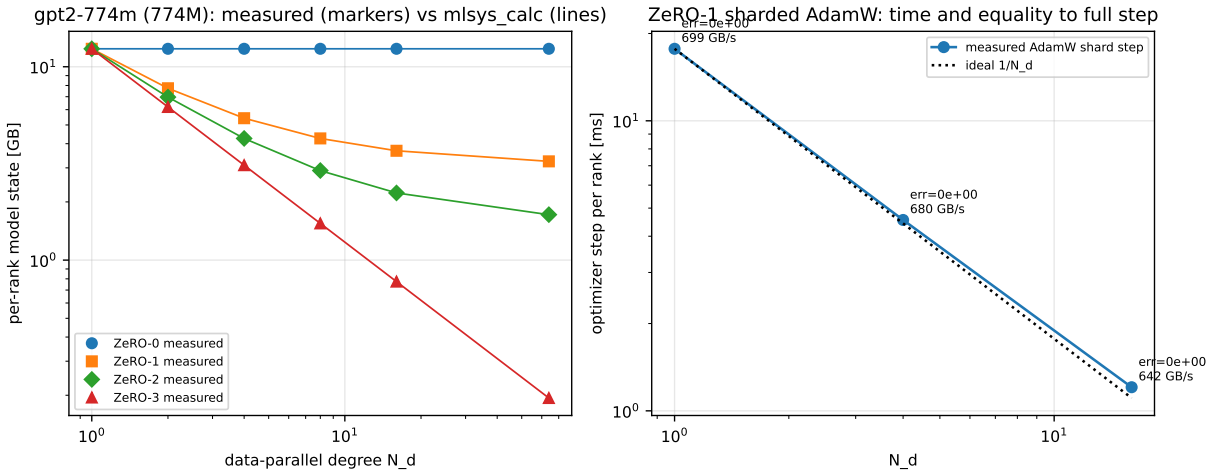


图 12: 左：774M 模型 rank 0 在 ZeRO-0..3 下真实分配的状态显存（点）与 `mlsys_calc`（线）；右：分片 AdamW 每 rank 的步时间与等价性。

表 5: rank 0 常驻模型状态（GB，`zero.jsonl`；N = 774,030,080，bf16 权重/梯度 + fp32 master/m/v）

N_d	ZeRO-0	ZeRO-1	ZeRO-2	ZeRO-3
1	12.384	12.384	12.384	12.384
8	12.384	4.260（模型 4.257）	2.905（2.903）	1.548（1.548）
64	12.384	3.241（3.241）	1.718（1.717）	0.194（0.194）

图 12 左与表 5：24 个 (N_d, stage) 组合的实测值与 `mlsys_calc.training_memory_bytes(N, dp, zero)` 的最大相对误差 0.099%，来自补齐到 N_d 倍数的 padding 与分配器取整。这个实验的价值不在“分配多少就是多少”，而在于把公式落到真实模型的扁平化参数上：ZeRO-1 在 $N_d = 8$ 时把 12.38 GB 压到 4.26 GB（−66%），已经能让 774M 在 8 GB 卡上放下状态；ZeRO-2 再省 1.35 GB；ZeRO-3 才让状态随 N_d 线性下降（$N_d = 64$ 时 0.194 GB），代价是前向/反向各一次

参数 all-gather。注意到 N_d 很大时 ZeRO-1/2 的下限分别是 $4N$ 与 $2N$ 字节（3.10 / 1.55 GB），这就是“大模型只能 ZeRO-3/FSDP [13]”的定量原因。

右图：对 $N_d \in \{1, 4, 16\}$ 逐 rank 执行分片 fused AdamW 两步并拼回，与整张量 AdamW **逐位相等**（max abs err = 0）——AdamW 是逐元素算子，分片不改变任何浮点运算顺序，这正是 ZeRO-1 无需改变收敛性的根据。每 rank 步时间 17.7 / 4.55 / 1.21 ms，与 $1/N_d$ 吻合。fused AdamW 每参数读 p, g, m, v 写 p, m, v 共 28 字节，$N_d = 1$ 时对应 $28 \times 774\text{M}/17.7\,\text{ms} = 1.22$ TB/s，约为实测拷贝带宽的 80%：优化器步是纯访存受限的，所以它的时间也按 ZeRO 的分片比例下降。

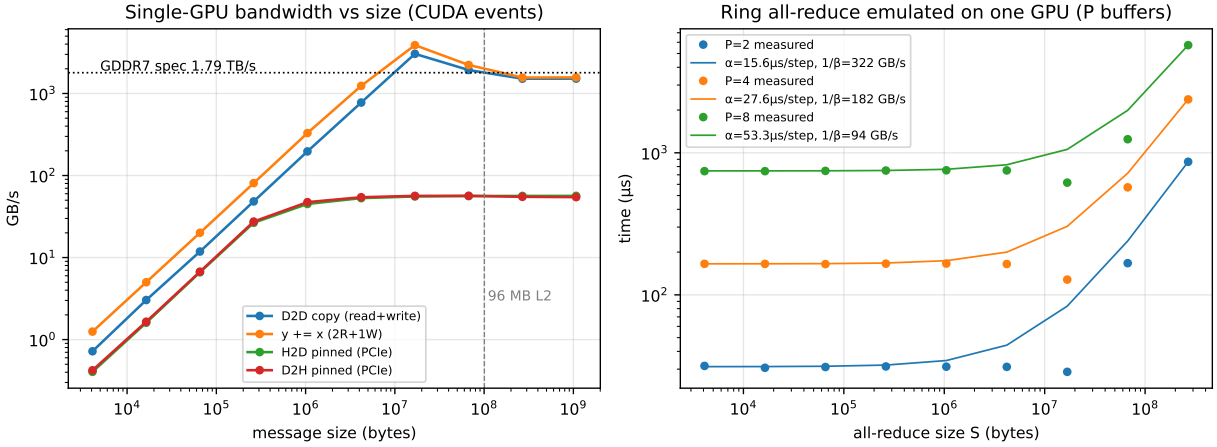


图 13: 左：单卡上四类“链路”的带宽随消息大小；右：单卡 ring all-reduce 仿真（P 个缓冲区）与 α-β 拟合。

图 13 左：D2D 拷贝在 16 MB 时达到 3.04 TB/s（读 + 写计），这是 96 MB L2 驻留的假象；超过 L2 后稳定在 1.52 TB/s（1 GiB），为 GDDR7 名义 1.79 TB/s 的 85%。pinned H2D/D2H 稳定在 56.5 / 54.5 GB/s，是 PCIe 5.0 x16 理论 64 GB/s 的 88%。所有曲线在 $\leq$ 64 KB 时都是一条斜率为 1 的直线：时间固定约 10 µs，即启动/同步开销 α。右图：单卡 ring 的时间在 $\leq$ 4 MB 时完全平坦（P=8：743 µs，等于 $2(P-1) \cdot P = 112$ 次启动 $\times$ 6.6 µs），拟合每步 α = 15.6 / 27.6 / 53.3 µs（P=2/4/8）。大消息端的“总线带宽”322 / 182 / 94 GB/s 只反映 P 份缓冲共用一块 HBM，不代表任何互连。数值正确性：与 `torch.stack(...).sum(0)` 的最大差 $\leq 2.9 \times 10^{-6}$（fp32 求和顺序不同）。

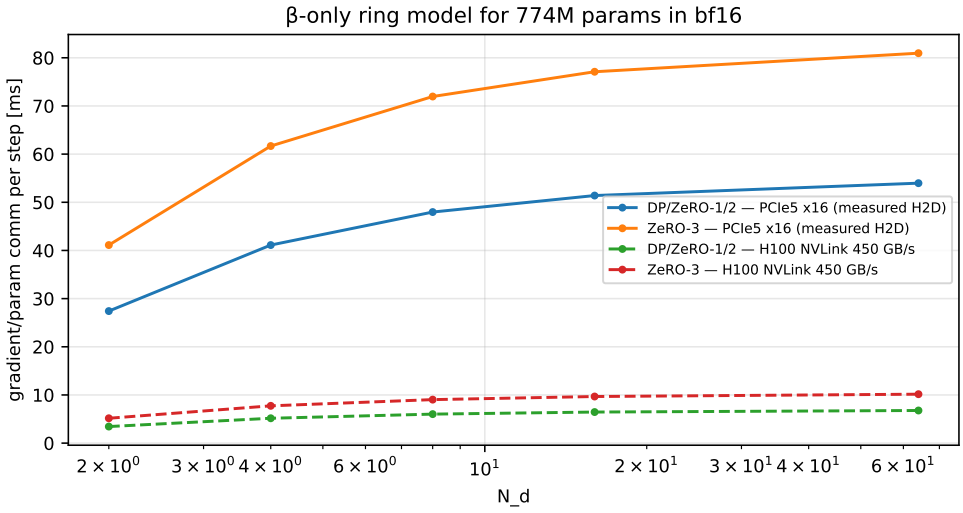


图 14: 用实测 PCIe 带宽与 H100 NVLink 参数化的 ZeRO 每步通信时间（774M，bf16，仅 β 项）。

图 14 把实测 β 代回 ZeRO 的通信量：DP/ZeRO-1/2 每步 2Ψ（RS+AG），ZeRO-3 3Ψ。在 $N_d = 8$ 的 PCIe 5090 机器上，774M 的 DP 每步 47.97 ms、ZeRO-3 71.96 ms；H100 NVLink 上 DP 只需 6.02 ms。结合单卡计算 145 ms/步（$b = 4$），PCIe 上 DP 的通信/计算比约 0.33，ZeRO-3 约 0.50——能否靠重叠隐藏，取决于反向里梯度分桶的粒度与 PCIe 是否与 H2D 数据加载争用。

结论：ZeRO

实测每 rank 状态与 $(2+2+12)N$ 的分片公式误差 $< 0.1\%$；分片 AdamW 与整张量逐位相等、时间按 $1/N_d$ 下降。单卡只能给出 $\alpha \approx 7$ µs/次启动与 HBM 侧 β；真正决定多卡 5090 训练的是 PCIe 的 56.5 GB/s，它让 DP 通信达到计算时间的 1/3。

4.6 GPU A4：真实 kernel 的 GPipe 与 1F1B

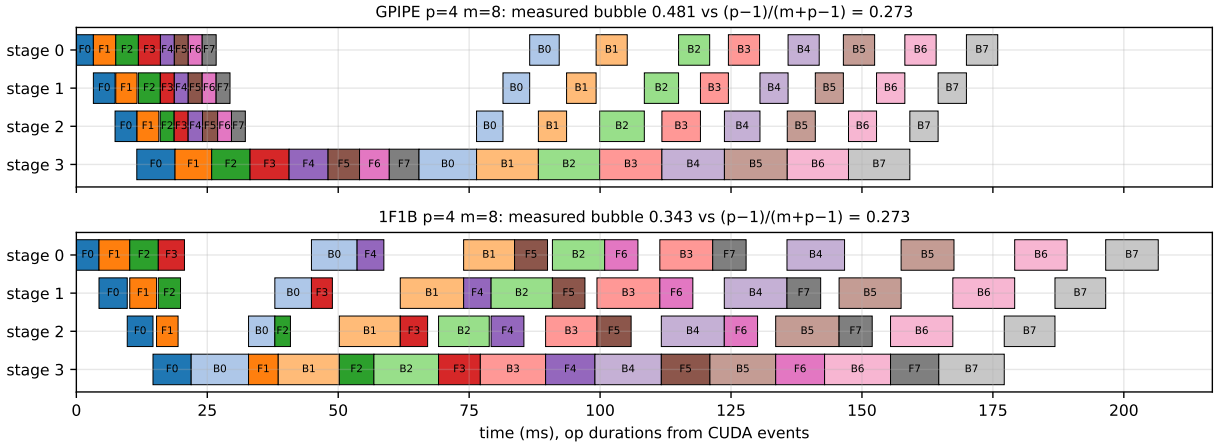


图 15: GPT-2 355M 切 4 级（每级 6 块；stage 0 含 embedding，stage 3 含 LM head + loss），micro-batch 4×512，$m = 8$。每个色块的长度是该算子的 CUDA event 实测时长，起止时刻按依赖关系重放。

表 6: 流水线实测（`pipeline.jsonl`；p=4；“不均衡模型” = $(m-1)T_{\max} + \sum_d T_d$，$T_d = t_f^d + t_b^d$ 取该次运行各级中位数）

调度	m	气泡实测	$(p-1)/(m+p-1)$	不均衡模型	makespan (ms)	峰值显存 (GB)	每级在途 mb
GPipe	1	0.750	0.750	0.750	45.5	3.77	[1,1,1,1]
GPipe	4	0.521	0.429	0.497	113.4	12.61	[4,4,4,4]
GPipe	8	0.481	0.273	0.502	175.9	24.40	[8,8,8,8]
GPipe	16			OOM（32 GB）			
1F1B	4	0.492	0.429	0.481	117.5	7.64	[4,3,2,1]
1F1B	8	0.343	0.273	0.354	206.6	7.64	[4,3,2,1]
1F1B	16	0.245	0.158	0.254	389.9	7.64	[4,3,2,1]
1F1B	32	0.233	0.086	0.203	723.9	7.64	[4,3,2,1]

图 15 and 表 6 给出三条结论。**(1) 在途激活：完全符合 1F1B 的论文命题。**每级在途 micro-batch 的峰值在 1F1B 下恰为 $[p-d] = [4,3,2,1]$，与 m 无关；GPipe 下为 $[m,m,m,m]$。单卡执行全部 4 级时的真实峰值显存：GPipe 从 3.77 GB（$m = 1$）涨到 24.40 GB（$m = 8$），$m = 16$ 时 OOM；1F1B 在 $m \geq 4$ 后恒为 7.64 GB。每级每 micro-batch 的激活实测为 613 / 605 / 605 / 1124 MB——stage 3 多出的约 520 MB 正是 $4 \cdot bsV = 4 \times 4 \times 512 \times 50257 = 412$ MB 的 fp32 log-softmax，加 bf16 权重副本。

（2）气泡：公式只在各级均衡时成立。实测气泡全部高于 $(p-1)/(m+p-1)$，而且 1F1B 在 $m=32$ 时停在 0.233 而不是 0.086。原因是各级不均衡：stage 3 的 t_f+t_b 约 21.9 ms，前三级约 16.5–18.4 ms（$m=32$ 的 1F1B 运行）。稳态下流水线的节拍由最慢级决定，makespan $\approx (m-1)T_{\max}+\sum_d T_d$，于是 $m\to\infty$ 时气泡趋向 $1-\bar{T}/T_{\max}$ 而不是 0。这个不均衡模型对 10 个运行的预测与实测相差 ≤ 0.03（GPipe $m=8$：176.3 vs 175.9 ms）。推论：工业界把 embedding/LM head 单独计入切分代价（Megatron 的 `--decoder-first/last-pipeline-num-layers`），否则加 micro-batch 也压不下气泡。

（3）GPipe 与 1F1B 的 makespan 相近，1F1B 并不更快。$m=8$ 时 GPipe 175.9 ms、1F1B 206.6 ms；理论上两者相同，差异来自算子时长的运行间波动（见下）。1F1B 的价值在显存：同样 32 GB，GPipe 最多 $m=8$（气泡 0.481），1F1B 可以用 $m=32$（气泡 0.233）。重放口径的等效吞吐分别是 93.1k 与 90.5k tokens/s，几乎相同——因为 1F1B 那几次运行的算子时长整体偏长（见下文的测量局限）；若用同一组算子时长，气泡从 0.481 降到 0.233 意味着吞吐提升 $(1-0.233)/(1-0.481)=1.48$ 倍。

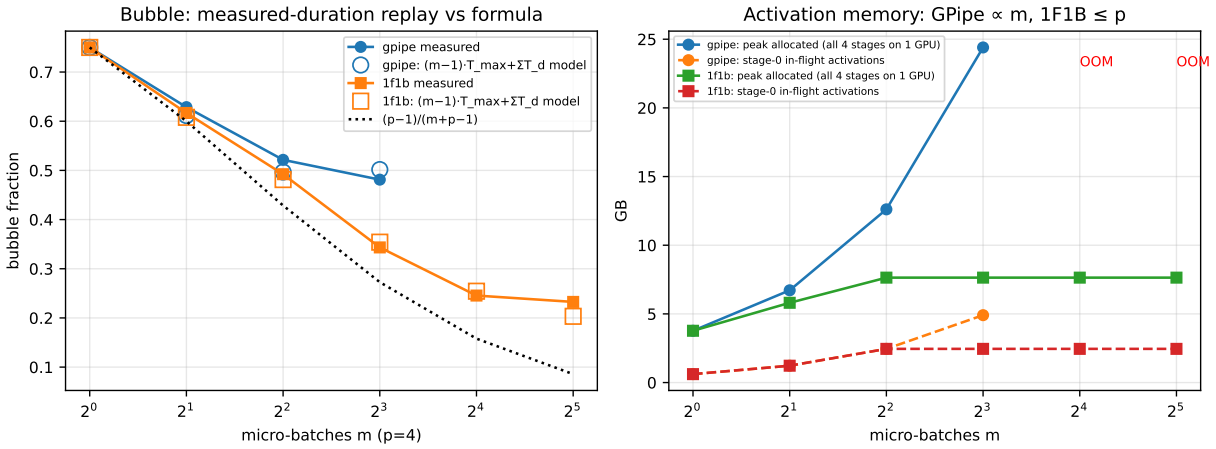


图 16: 左：气泡随 m（实线实测、空心为不均衡模型、点线为均衡公式）；右：峰值显存与 stage 0 的在途激活。

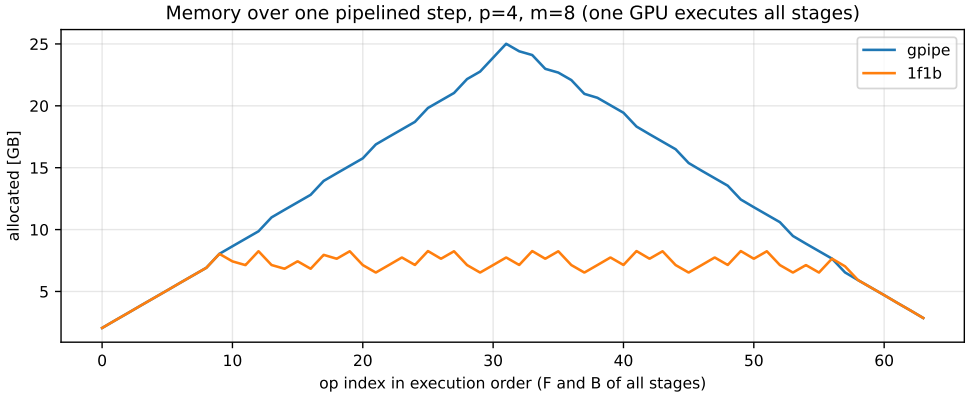


图 17: 一次流水线步内的显存轨迹（$m=8$）：GPipe 先把 32 个前向的激活全部堆起来，1F1B 在稳态里每做一个 F 就释放一个 B。

图 17 直观展示了二者显存行为的差别：GPipe 的轨迹是“先涨到顶再降”的三角形，1F1B 是一段平台。

测量口径的局限。 单卡无法让 4 级真正并行，Gantt 图是“真实算子时长 + 依赖重放”，不包含 p2p 通信，也不包含多卡上算子并发时的相互干扰。另外同一个算子在不同运行里的时长差别明显（如 stage 0 前向：GPipe $m = 8$ 中位 2.96 ms，1F1B $m = 8$ 为 5.66 ms）：这些算子只有 6 层、2048 token，GPU 时间约 2–3 ms，CPU 只有 2 个受 cgroup 配额限制的核，当 CPU 下发跟不上时 CUDA event 之间会混入 GPU 空等。因此本节的“绝对毫秒”只作量级参考，但每次运行内部用同一组时长做重放，气泡与模型的比较是自洽的。

结论：流水线

1F1B 的在途激活实测恰为 $[4, 3, 2, 1]$，峰值显存与 m 无关（7.64 GB），GPipe 在 $m = 16$ 时 OOM；气泡公式 $(p-1)/(m+p-1)$ 假设各级均衡，LM head 让最后一级重 20–30%，实测气泡因此停在 0.23 附近，用 $(m-1)T_{\max} + \sum T_d$ 可预测到 ± 0.03。

5 与理论模型的对照

5.1 静态显存：16N 的三种拆法

设参数数为 N。Adam 需要每参数两个一阶/二阶矩 m, v。

$$\text{fp32:} \quad 4N_{\text{w}} + 4N_{\text{g}} + 8N_{m,v} = 16N \tag{1}$$

$$\text{AMP (fp32 主权重即模型权重):} \quad 4N + 4N + 8N = 16N \tag{2}$$

$$\text{bf16 + fp32 master（ZeRO/Megatron）:} \quad 2N + 2N + (4+4+4)N = 16N \tag{3}$$

$$\text{纯 bf16:} \quad 2N + 2N + 4N = 8N \tag{4}$$

`mlsys_calc.training_memory_bytes` 实现的是式 (3)，其分项（2/2/12）只与 bf16+master 方案逐项吻合：774M 上实测 1.58/1.57/9.47 GB，模型 1.55/1.55/9.29 GB（N=774,030,080 时 $2N = 1.548$ GB，$12N = 9.288$ GB），差异 1–2% 为分配器取整与小张量。AMP 的总量同为 16N，但分项是 4/4/8——面试里说“混合精度省一半显存”是错的：省的是激活和带宽，不是静态状态。

5.2 激活：逐张量计数

对一个 pre-LN GPT 块（无 dropout，flash attention），反向需要保存的张量（以 sbh 个元素为单位）是：LN1 输入 1、QKV 线性的输入 1、q, k, v 共 3、attention 输出 1（它同时是输出投影的输入：flash kernel 的输出缓冲布局是 $[B, T, H, D]$，`transpose+reshape` 是 view，不复制）、LN2 输入 1、FC 输入 1、GELU 输入 4、FC2 输入（GELU 输出）4，共 16 个。于是每层

$$A_{\text{layer}} = c \cdot sbh, \qquad c = \begin{cases} 16 \times 4 = 64 & \text{fp32} \\ 16 \times 2 = 32 & \text{纯 bf16} \\ 32 + 2 \times 2 = 36 & \text{AMP（残差流与两个 LN 输入为 fp32）} \end{cases} \tag{5}$$

再加 cross-entropy 保存的 fp32 log-softmax 输出 $4bsV$（bf16 logits 本身不被保存），以及 AMP 为反向保存的 bf16 权重副本约 $2N$。Korthikanti 等 [10] 给出 fp16 下每层 $sbh(34 + 5as/h)$：attention 11（QKV 输入 2、Q, K 4、V 2、投影输入 2、dropout 掩码 1）、MLP 19（输入 2、GELU 输入 8、GELU 输出 8、掩码 1）、两个 LN 4，以及注意力分数/softmax/dropout 的 $5as^2b$。去掉两个 1 字节 dropout 掩码、并且 flash attention 不保存 s^2 项后正好是 $34 - 2 = 32$，与本文纯 bf16 的 $c = 32$ 逐项对应。最初我们把 attention 输出与投影输入当作两个张量（17 个），得到 68/38/34，系统性高估约 6%；实测把它纠正为 16 个——“测量反过来修正模型”的一个小例子。

5.3 训练步时间与 MFU

每 token 训练 FLOPs $\approx 6N+12Lhs$。774M、$s=1024$：$6N=4.643\times10^9$，$12Lhs=0.566\times10^9$，合计 5.209×10^9。纯 bf16 实测 28.2k tokens/s，得 1.47×10^{14} FLOP/s，即名义峰值的 70.2%。这一水平与 Megatron/PaLM 报告的大模型 MFU（40–60%）相比偏高，原因是单卡没有任何通信、且 GPT-2 774M 的 GEMM（$h=1280$，$4h=5120$）已足够大。

5.4 α-β 模型的单卡定参

Ring all-reduce 时间 $T=2(P-1)\alpha+2\frac{P-1}{P}S\beta$。单卡 ring 仿真里每步有 P 次 kernel 启动，小消息平台给出每步 α：$P=2,4,8$ 分别 15.6、27.6、53.3 µs，即每次启动约 6.7–7.8 µs（与 4 KB 拷贝的 10–11 µs 事件计时一致）。大消息端的“总线带宽”322/182/94 GB/s 随 P 下降，因为所有“链路”共享同一块 HBM：P 个 rank 同时搬运，单卡 HBM 1.52 TB/s 被 P 份分摊，再除以拷贝 + 加法的 3 次访存。所以单卡仿真能测 α（启动/同步开销）和算法正确性（误差 $\le 2.9\times10^{-6}$，fp32 求和顺序），但**不能**测互连 β。真正可用的 β 是 PCIe：pinned H2D 56.5 GB/s、D2H 54.5 GB/s（1 GiB）。代入 774M、bf16 梯度（$2N=1.548$ GB）、$N_d=8$：

$$T_{\text{DP}}=2\cdot\tfrac{7}{8}\cdot\frac{1.548\,\text{GB}}{56.5\,\text{GB/s}}=47.9\,\text{ms},\qquad T_{\text{ZeRO-3}}=1.5\,T_{\text{DP}}=71.9\,\text{ms}\tag{6}$$

而 774M 单步计算（$b=4,s=1024$）约 145 ms（表 3，28.2k tokens/s）。这意味着在 8 卡 PCIe 5090 上做 DP 时，通信约占计算的 1/3，必须靠梯度分桶与反向重叠才能隐藏；ZeRO-3 的额外 0.5 倍通信几乎无法隐藏。同样的量在 H100 NVLink（450 GB/s）上是 6.0 ms。

5.5 流水线气泡：均衡与不均衡

均衡时每级前 + 反向耗时 T，GPipe/1F1B 的 makespan 为 $(m+p-1)T$，有效工作 mpT，故气泡 $=(p-1)/(m+p-1)$（`mlsys_calc.pipeline_bubble`）。不均衡时稳态节拍取最慢级 $T_{\max}$：第一个 micro-batch 穿过全部级需要 $\sum_d T_d$，此后每 $T_{\max}$ 出一个，

$$\text{makespan}\approx(m-1)\,T_{\max}+\sum_d T_d,\qquad \text{bubble}=1-\frac{m\sum_d T_d}{p\,[(m-1)T_{\max}+\sum_d T_d]}\xrightarrow{m\to\infty}1-\frac{\bar T}{T_{\max}}.\tag{7}$$

代入 1F1B $m=32$ 的实测 T_d = 16.5/18.2/18.4/21.9 ms，得极限气泡 $1-18.7/21.9=0.14$，有限 m 时 0.203，实测 0.233。

5.6 重计算的时间与显存

全量重计算（每块 checkpoint）时，反向前要把每块的前向再做一遍：$T=F+B+F$，取 $B\approx2F$ 得 $T/T_0=4/3$。显存上只保留每块输入（AMP 下残差流为 fp32，$4sbh$ 字节/层）加上“正在重算的那一块”的完整激活：

$$A_{\text{full}}\approx L\cdot4sbh+c\cdot sbh+4bsV,\qquad A_{\text{none}}=L\cdot c\cdot sbh+4bsV\ (+2N).\tag{8}$$

774M（$L=36$, $c=36$）时块内激活压缩到约 $4/36=1/9$，但 logits 项 $4bsV$ 不受影响——这就是全量重计算后激活不会降到 1/9 的原因。选择性重计算（只 checkpoint MLP）去掉每层 c 中 MLP 的 $2+8+8=18$ 字节（AMP 下 FC 输入 2、GELU 输入 8、GELU 输出 8），时间代价只是一次 MLP 前向（约占块 FLOPs 的 2/3）；只 checkpoint attention 子层则只省 $2+6+2=10$ 字节（子层输入的 4 字节 fp32 残差作为 checkpoint 输入仍要保存），而 flash attention 下注意力部分的 FLOPs 本来就小。两种选择性方案还会省掉对应线性层的 autocast bf16 权重副本（MLP $8h^2\cdot2$、attention $4h^2\cdot2$ 字节/层），实测见表 4。

5.7 $\sqrt{n}$ 与 Checkmate

Chen 2016 的 $\sqrt{n}$ 分段把 n 层的峰值从 $O(n)$ 降到 $O(\sqrt{n})$，代价是一次额外前向；Checkmate 用 MILP 在异构图上求最优。本 lab 的 CPU DP 在异构 32 层链上比 $\sqrt{n}$ 少 27% 重算；GPU 实验里我们只比较了“每块/隔块/子层”这类工程上最常见的策略，它们对应 Megatron 的 `full` 与 `selective` 两档。

6 面试专题

面试追问：训练一个 7B 模型，Adam 混合精度，需要多少显存？

静态状态 $16\Psi = 112$ GB（bf16 权重 2 + bf16 梯度 2 + fp32 master/m/v 12），还没算激活。本文 774M 实测 16.31 B/参数（bf16+master），与公式差 2%。单卡 80 GB 放不下，需要 ZeRO-1 起步：$N_d = 8$ 时 $(4 + 12/8)\Psi = 38.5$ GB。

常见错答：“bf16 训练，每参数 2 字节，14 GB”——漏了梯度和优化器；或“混合精度比 fp32 省一半”——静态状态一样是 16 字节，省的是激活和带宽。

面试追问：激活显存怎么估？为什么你的公式是 32 而 Megatron 论文是 34？

每层 $c \cdot sbh$ 字节：逐个数反向要保存的张量。纯 bf16、无 dropout、flash attention 时为 16 个 sbh 张量 = 32 字节；Megatron 的 34 多了两个 dropout 掩码，$5as/h$ 是没有 flash 时保存的注意力分数。再加 loss 的 $4bsV$。本文 12 个配置实测/模型 = 1.004–1.013。

常见错答：忘掉 logits 项——GPT-2 124M、V=50257 时它占每 token 激活的 38%；或以为 AMP 激活是纯 bf16 的一半（残差流仍是 fp32，36 vs 32）。

面试追问：activation checkpointing 的时间代价是多少？

全量重计算理论 $4/3$（多一次前向）。774M 实测只慢 18.9%，因为 LM head、优化器步和比 $2F$ 更贵的反向都不被重算；激活降到 19.8%，最大 batch 从 8 到 24。只重算 MLP：慢 9.3%、省 46% 激活。注意 GPU 已饱和（MFU 72%）时更大的 batch 不会提高吞吐，$b = 24$ 反而只有 24.7k vs 28.9k tokens/s。

常见错答：“重计算是 2 倍时间”（把前向 + 反向都重做）；或“checkpoint 后激活就只剩 $1/L$”——logits 项与正在重算的那一块不受影响。

面试追问：ZeRO-1/2/3 各省什么、通信多多少？

ZeRO-1 分片 12Ψ 优化器状态，ZeRO-2 再分片梯度，通信都与 DP 相同（RS+AG = 2Ψ）；ZeRO-3 再分片参数，前向、反向各多一次 all-gather，通信 3Ψ（1.5 倍）。本文实测 $N_d = 8$ 时每 rank 12.38 / 4.26 / 2.91 / 1.55 GB。分片 AdamW 与整张量逐位相等，因为它是逐元素运算。

常见错答：“ZeRO-2 通信更多”——梯度 reduce-scatter 本来就是 all-reduce 的前半段，ZeRO-2 只是不再做 all-gather 梯度。

面试追问：没有 NVLink 的 8 卡 5090 上做 DP，通信瓶颈多大？

实测 PCIe 5.0 pinned 拷贝 56.5 GB/s。774M bf16 梯度 1.548 GB，ring all-reduce $2 \cdot \frac{7}{8} \cdot 1.548/56.5 = 48$ ms，单卡计算 145 ms/步（$b = 4$），通信占 1/3；ZeRO-3 72 ms。H100 NVLink 上同样的量是 6 ms。所以消费级多卡训练要么加大每卡 batch 拉长计算、要么用梯度累积降低同步频率。

常见错答：用 HBM 带宽或单卡 ring 仿真的“带宽”去估多卡通信——单卡 ring 的 94–322 GB/s 只是 P 份缓冲共享 HBM 的结果。

面试追问：1F1B 比 GPipe 快吗？

不快。均衡时两者气泡都是 $(p-1)/(m+p-1)$。1F1B 把在途激活从 m 降到 $\le p$：实测每级 $[4,3,2,1]$，峰值 7.64 GB 与 m 无关，GPipe $m=16$ 就 OOM。省下的显存允许更大的 m，间接降低气泡。真正降低气泡的是交错调度（v 倍），代价是 v 倍 p2p 次数和更多在途激活。

常见错答：“1F1B 气泡更小”；或认为加 micro-batch 能让气泡趋于 0——实测各级不均衡时极限气泡是 $1-\bar{T}/T_{\max}$。

面试追问：怎么算 MFU？你的 70% 可信吗？

MFU = tokens/s $\times(6N+12Lhs)$ / 峰值。774M 纯 bf16：28.2k tokens/s $\times 5.209\times10^9$ = 147 TFLOP/s，对 209.5 名义峰值为 70.2%。但实测 bf16 GEMM 能到 234.7 TFLOP/s（高于名义），以此为分母是 62.7%。面试中要说明分母口径；且单卡无通信，多卡 MFU 会更低。

常见错答：用 $6N$ 而忽略 $12Lhs$（$s=1024$ 时少算 11%）；或拿稀疏峰值当分母。

面试追问：为什么 TP 一般限制在单机内？

Megatron 每个 Transformer 块前向 2 次、反向 2 次 all-reduce，消息 $b\cdot s\cdot h$ 与 TP 度无关，且都在关键路径上。跨机 IB（50 GB/s/端口）约为 NVLink 的 1/9；PCIe 机器上更糟。CPU lab 中 Llama-3-8B、TP=8、seq=8192 每步通信 42.4 ms（模型值）。

常见错答：“TP 通信量随 TP 度增大”——每次 all-reduce 的消息大小不变，变的是每卡计算量缩小，通信占比上升。

面试追问：all-reduce 什么时候用 ring，什么时候用 tree？

时间 = 步数 $\cdot\alpha$ + 字节 $\cdot\beta$。ring 字节最优 $2(P-1)/P\cdot S$，步数 $2(P-1)$；tree/halving-doubling 步数 $O(\log P)$。CPU lab 中 $P=8$、$\alpha=5$ µs 时交叉点约 4.27 MB。本文单卡实测每次 kernel 启动约 7 µs，说明 α 的下界就是 launch 开销，P 大时 ring 的 $2(P-1)\alpha$ 不可忽视。

常见错答：“ring 总是最优”；“halving-doubling 总比 ring 好”——物理环拓扑上它因链路争用慢 3 倍。

20 秒电梯陈述。 “我在 RTX 5090 上把训练显存逐组件测了一遍：fp32、AMP、bf16+fp32 master 三种布局静态显存都是 16 字节/参数，误差 2% 以内；自己逐张量推的激活公式和实测差 1% 以内，还因此修正了一个 attention 输出共享存储的细节。774M 单卡 MFU 70%。流水线上 1F1B 的在途激活实测正好是 4、3、2、1，峰值和 micro-batch 数无关；气泡公式在最后一级带 LM head 时偏差很大，我用最慢级节拍模型把误差压到 0.03。”

STAR 故事：激活模型为什么偏了 6%。 **S**：要验证 GPT-2 训练显存模型，表格里 12 个配置的激活实测都比我的逐张量模型低约 6%。**T**：判断是测量问题还是模型问题，而不是加一个拟合系数糊过去。**A**：先排除分配器——纯 bf16 与 fp32 的偏差比例一样，说明不是取整；再按 batch×seq 扫描，偏差严格正比于 $L\cdot sbh$，说明每层恰好多算了一个 sbh 张量；逐个检查保存的张量，发现 flash attention 的输出缓冲是 $[B,T,H,D]$ 布局，`transpose+reshape` 是 view，输出投影保存的输入与 attention 输出共享存储。**R**：每层从 17 个改为 16 个张量（fp32/AMP/bf16 = 64/36/32 字节），12 个配置的实测/模型从 0.94 变为 1.004–1.013，并与 Megatron 的 34 减去 dropout 掩码逐项对上。

7 局限

不能推出的结论

- **单卡不是集群**：没有测到任何真实互连。单卡 ring 仿真的“带宽”是 HBM 被 P 份缓冲共享的结果；多卡结论只来自代入 PCIe 实测 β 的模型。
- **ZeRO 是显存核算 + 优化器等价性**，不是完整的 ZeRO-3 训练（没有按层 all-gather / 预取 / 与计算重叠）；不能用本文数字说明 ZeRO-3 的端到端吞吐。
- **流水线是“实测时长 + 依赖重放”**：4 级在同一卡上串行执行，没有 p2p 通信与并发干扰；2 核 CPU 的下发抖动让同一算子在不同运行中相差可达 2 倍，绝对毫秒只作量级参考。
- **纯 bf16 的 8 B/参数**只说明显存；bf16 的 7 位尾数会让 $|\Delta w| < 2^{-8}|w|$ 的更新被舍入掉，本文未做收敛实验，不能推出“纯 bf16 可替代 fp32 master”。
- **MFU 只有单卡口径**，且分母是 209.5 TFLOP/s 名义值；以实测 GEMM 峰值为分母低约 11%。
- **随机初始化、随机 token**：显存与时间与数据无关，但不能用于任何收敛或精度结论。
- GPU 被 7 个代理分时共享：计时都在互斥锁内，但 CPU 配额共享，影响小算子与启动受限的测量。

8 结论

1. 16N 规则在 fp32、AMP、bf16+fp32 master 三种布局下均成立（16.00–16.32 B/参数），纯 bf16 为 8 B/参数；`mlsys_calc` 的 2/2/12 分项与 bf16+master 逐项吻合。
2. 激活 $= L \cdot c \cdot sbh + 4bsV\,(+2N)$，$c$ = 64/36/32（fp32/AMP/bf16），与实测误差 $\le 1.3\%$，对 $b \cdot s$ 严格线性；词表大的小模型里 logits 项占 38%。
3. 774M 纯 bf16 单卡 MFU 70.2%（名义）/ 62.7%（实测 GEMM）。
4. ZeRO 各 stage 每 rank 状态与公式误差 $< 0.1\%$；分片 AdamW 逐位等价、时间 $\propto 1/N_d$；多卡 5090 的 DP 通信由 PCIe 56.5 GB/s 决定，约为计算的 1/3。
5. 1F1B 的在途激活 $[4, 3, 2, 1]$ 与峰值显存 7.64 GB 与 m 无关；气泡公式需要按最慢级修正。
6. 每块全量重计算：激活降到 19.8%、最大 batch 8→24，时间 +18.9%（低于理论 33%）；只重算 MLP：+9.3% 时间、-46% 激活；节省量与逐张量模型（含 autocast 权重副本）差 $\le 0.8\%$。GPU 已饱和时更大 batch 不能抵消重算开销（第 4.4 节）。

参考文献

[1] R. Thakur, R. Rabenseifner, W. Gropp. Optimization of Collective Communication Operations in MPICH. *IJHPCA*, 2005.

[2] P. Patarasuk, X. Yuan. Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *JPDC*, 2009.

[3] M. Shoeybi et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053, 2019.

[4] S. Rajbhandari et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *SC*, 2020.

[5] Y. Huang et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *NeurIPS*, 2019.

[6] D. Narayanan et al. PipeDream: Generalized Pipeline Parallelism for DNN Training. *SOSP*, 2019.

[7] D. Narayanan et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *SC*, 2021.

[8] T. Chen et al. Training Deep Nets with Sublinear Memory Cost. arXiv:1604.06174, 2016.

[9] P. Jain et al. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. *MLSys*, 2020.

[10] V. Korthikanti et al. Reducing Activation Recomputation in Large Transformer Models. arXiv:2205.05198, 2022.

[11] P. Micikevicius et al. Mixed Precision Training. *ICLR*, 2018.

[12] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.

[13] Y. Zhao et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *VLDB*, 2023.